SPÉCIFICATION TECHNIQUE & FONCTIONNELLE

Migration WPF → ASP.NET MVC

Page principale — Tableau de données éditable en ligne avec DataTables.js

Document de référence pour l'équipe de développement

Version 1.0

Sommaire

1. Contexte et objectif
2. Périmètre fonctionnel
    1. Affichage du tableau
    2. Colonne « Customer » composite
    3. Édition en ligne (inline edit)
    4. Boutons de validation / annulation
    5. Bouton « Sauvegarder les changements »
3. Maquette fonctionnelle (wireframe)
4. Architecture technique proposée
5. Modèle de données / ViewModel
6. Implémentation DataTables (front-end)
    1. Structure HTML
    2. CSS — rendu de la colonne Customer
    3. Initialisation DataTables
    4. Passage en mode édition
    5. Gestion des boutons Valider / Annuler
    6. Sauvegarde globale
7. Implémentation ASP.NET MVC (back-end)
    1. Contrôleur
    2. Endpoints AJAX
8. Règles de gestion & cas limites
9. Checklist de recette

1. Contexte et objectif

L'application WPF existante expose, sur sa page principale, un tableau de données permettant à l'utilisateur de consulter des lignes, d'éditer certaines d'entre elles directement dans le tableau (édition « inline »), puis de valider ou d'annuler chaque modification ligne par ligne, avant d'enregistrer l'ensemble des changements en base de données via un bouton global.

L'objectif de ce document est de décrire précisément le comportement attendu afin de reproduire à l'identique cette expérience utilisateur dans la nouvelle application web **ASP.NET MVC**, en s'appuyant sur la librairie JavaScript **DataTables** pour l'affichage et l'interaction avec le tableau.

> **OBJECTIF CLÉ**
> Offrir une expérience d'édition en ligne équivalente à celle du WPF (édition directe dans la cellule, apparition dynamique des boutons de validation, sauvegarde différée/groupée), tout en restant dans les standards d'une application web MVC (AJAX, JSON, séparation contrôleur / vue).

2. Périmètre fonctionnel

2.1 Affichage du tableau

- Le tableau affiche la liste des enregistrements (ex. commandes / lignes clients) avec pagination, tri par colonne et recherche globale — fonctionnalités natives de DataTables.
- Chaque ligne représente un enregistrement identifié par une clé unique (`Id`).
- Certaines colonnes sont **éditables**, d'autres restent en **lecture seule** (à définir avec le métier / repris de la grille WPF existante).
- Une colonne « Actions » est réservée, en fin de ligne, à l'apparition conditionnelle des boutons Valider (✓) et Annuler (×).

2.2 Colonne « Customer » composite

La colonne **Customer** affiche **trois informations distinctes dans une seule cellule**, chacune avec un alignement horizontal spécifique à l'intérieur de la cellule :

Valeur	Alignement dans la cellule	Exemple
ClientId	Gauche (bord gauche de la cellule)	`10423`
Code client	Centre *(centré horizontalement dans la cellule)*	`CL-PARIS-01`
Currency	Droite (bord droit de la cellule)	`EUR`

Visuellement, dans une même ligne de cellule, on retrouve donc l'identifiant à gauche, le code client au centre, et la devise à droite — sans que ces trois informations ne soient dans des colonnes séparées.

Id	Customer			Montant	Date	Actions
1	10423	CL-PARIS-01	EUR	1 250,00	04/11/2026	

2	10987	CL-LYON-07	USD	980,50	05/11/2026	✓ ✕
3	10311	CL-NANTES-02	EUR	430,00	03/11/2026	

Figure 1 — Wireframe : la ligne 2 est en cours d'édition (fond jaune pâle), les boutons Valider / Annuler sont visibles uniquement sur cette ligne.

2.3 Édition en ligne (inline edit)

- Au clic (ou double-clic, à valider avec le métier) sur une cellule éditable, celle-ci se transforme en champ de saisie (`<input>` , `<select>` … selon le type de donnée).
- Plusieurs cellules de la même ligne peuvent être éditées avant validation (comme dans le WPF, qui autorise la modification de plusieurs champs avant d'enregistrer la ligne).
- Tant qu'aucune saisie n'a été effectuée sur la ligne, aucun bouton n'est visible.

2.4 Boutons de validation / annulation

RÈGLE DÉCLENCHANTE

Les boutons **✓ (Valider)** et **× (Annuler)** apparaissent dans la colonne Actions **dès qu'une saisie est détectée** (évènement `input` / `change`) sur un des champs éditables de la ligne — exactement comme dans l'application WPF.

Bouton	Icône	Comportement
Valider	✓ (vert)	Marque la ligne comme « modifiée » (état local, en mémoire côté client), fige/affiche les nouvelles valeurs, masque les boutons. *Aucun appel serveur immédiat* : l'enregistrement en base ne se fait qu'au clic sur le bouton global « Sauvegarder les changements » (cf. §2.5), pour reproduire le fonctionnement WPF où la validation ligne par ligne prépare la donnée avant un enregistrement groupé.
Annuler	× (rouge)	Restaure la/les valeur(s) d'origine de la ligne (avant édition), sort du mode édition, masque les boutons, retire la ligne de la liste des lignes modifiées si elle y figurait.

POINT À VALIDER AVEC LE MÉTIER

Dans le WPF, le bouton ✓ déclenche-t-il un enregistrement immédiat en base pour la ligne concernée, ou se contente-t-il de « confirmer » la saisie localement en attendant le bouton global ? Ce document part de la seconde hypothèse (cohérente avec la présence d'un bouton « Sauvegarder les changements » global), à confirmer avant développement.

2.5 Bouton « Sauvegarder les changements »

- Bouton unique, visible en permanence (ou activé uniquement si au moins une ligne a été validée localement — à définir).
- Au clic, l'ensemble des lignes marquées comme « modifiées » (validées via ✓) sont envoyées en un seul appel AJAX au contrôleur MVC, pour un enregistrement groupé en base de données.
- En cas de succès : notification de confirmation, réinitialisation de l'état « modifié » des lignes, rechargement ou mise à jour des données du tableau.
- En cas d'échec (partiel ou total) : affichage des erreurs, conservation des lignes en échec dans l'état « modifié » afin de ne pas perdre la saisie utilisateur.

3. Architecture technique proposée

Couche	Technologie	Rôle
Vue	Razor (.cshtml)	**Rendu complet des lignes du tableau côté serveur**, via une boucle `@@foreach` sur `@@Model.Site` (le contrôleur alimente directement le Model de la vue, pas d'appel AJAX au chargement)
Front-end	DataTables.js + jQuery + JS/CSS custom	Tri, pagination, recherche sur le tableau déjà présent dans le DOM, édition inline, gestion des boutons
Échanges	AJAX (JSON)	Envoi des modifications uniquement (bouton « Sauvegarder les changements »), pas de chargement initial
Contrôleur	ASP.NET MVC Controller	`Index` (charge `Model.Site` et retourne la vue), `SaveChanges` (enregistrement groupé)
Accès données	Entity Framework / Dapper (selon existant)	Lecture / écriture base de données

POINT IMPORTANT

Contrairement à un DataTables « full AJAX » classique, ici **les données ne sont pas chargées en JSON au démarrage** : elles sont injectées directement dans le HTML par le moteur Razor via `@@foreach (var item in Model.Site)` , exactement comme la grille WPF était alimentée par son `ItemsSource` . DataTables est initialisé **sur un tableau déjà rempli** (mode « DOM sourcing »), et sert uniquement aux fonctions de tri / pagination / recherche / édition inline. Seul l'enregistrement des modifications (bouton global) transite en AJAX.

4. Modèle de données / ViewModel

Le Model de la vue `Index.cshtml` expose la collection `Site` consommée par le `@@foreach` ; c'est cette même structure qui sert de base aux lignes du tableau et à la logique d'édition inline côté JavaScript (via les attributs `data-*` posés sur chaque `<tr>` , cf. §5.1).

```
// Modèle de page (passé à la vue Index)
public class CustomersPageViewModel
{
    public List<CustomerRowViewModel> Site { get; set; }
}

// Une ligne du tableau
public class CustomerRowViewModel
{
    public int Id { get; set; }

    // Composantes de la colonne "Customer"
    public int ClientId { get; set; }
    public string CustomerCode { get; set; }
    public string Currency { get; set; }

    public decimal Montant { get; set; }
    public DateTime Date { get; set; }

    // autres colonnes éditables reprises de l'existant WPF...
}

// DTO utilisé pour l'enregistrement groupé (AJAX de sauvegarde uniquement)
public class SaveChangesRequest
{
    public List<CustomerRowViewModel> ModifiedRows { get; set; }
}
```

5. Implémentation DataTables (front-end)

5.1 Structure HTML / Razor

Le tableau est entièrement rendu côté serveur : la vue parcourt `Model.Site` en Razor et génère une ligne `<tr>` par enregistrement. Chaque ligne porte un attribut `data-id` (clé de l'enregistrement) qui servira de repère au JavaScript pour l'édition et la sauvegarde — DataTables n'est initialisé qu'**après** que ce HTML soit déjà présent dans le DOM.

```
<table id="customersTable" class="table table-striped" style="width:100%">
  <thead>
    <tr>
      <th>Id</th>
      <th>Customer</th>
      <th>Montant</th>
      <th>Date</th>
      <th>Actions</th>
    </tr>
  </thead>
  <tbody>
    @@foreach (var item in Model.Site)
    {
        <tr data-id="@@item.Id">
            <td>@@item.Id</td>

            <td>
                <span class="customer-cell">
                    <span class="client-id">@@item.ClientId</span>
                    <span class="customer-code">@@item.CustomerCode</span>
                    <span class="currency">@@item.Currency</span>
                </span>
            </td>

            <td class="editable-cell" data-field="Montant">
                <span class="cell-value">@@item.Montant</span>
                <input type="text" class="cell-input d-none"
                       value="@@item.Montant" data-original="@@item.Montant" />
            </td>

            <td>@@item.Date.ToString("dd/MM/yyyy")</td>

            <td class="actions-cell">
                <span class="btn-validate d-none" title="Valider">&#10003;</span>
                <span class="btn-cancel d-none" title="Annuler">&#10005;</span>
            </td>
        </tr>
    }
  </tbody>
</table>

<button id="btnSaveAll" class="btn btn-primary" disabled>
  Sauvegarder les changements
</button>
```

> **ATTENTION**
>
> Chaque cellule éditable contient à la fois le texte affiché (`.cell-value`) et un champ de saisie masqué (`.cell-input` , avec sa valeur d'origine mémorisée dans `data-original`) directement rendus par Razor. Il n'y a donc plus de `render()` JavaScript côté DataTables pour construire ce HTML : il est déjà là au chargement de la page.

5.2 CSS — rendu de la colonne Customer

La cellule « Customer » utilise un conteneur en `position: relative` et trois éléments positionnés en `absolute` (gauche / centré / droite) afin de respecter l'alignement demandé à l'intérieur d'une seule et même cellule.

```
.customer-cell {
    position: relative;
    display: block;
    width: 100%;
    min-height: 20px;
}
.customer-cell .client-id {
    position: absolute;
    left: 0;
    text-align: left;
}
.customer-cell .customer-code {
    position: absolute;
    left: 50%;
    transform: translateX(-50%);
    text-align: center;
}
.customer-cell .currency {
    position: absolute;
    right: 0;
    text-align: right;
}
```

5.3 Initialisation DataTables (mode « DOM sourcing »)

Le tableau étant déjà rempli par Razor, DataTables est initialisé **sans option `ajax` ni `columns.render`** : il se contente de « prendre la main » sur le HTML existant pour activer le tri, la pagination et la recherche. On désactive le tri sur la colonne Actions, et on précise un tri personnalisé sur la colonne Customer (puisqu'elle contient trois valeurs dans une seule cellule).

```
const table = $('#customersTable').DataTable({
    paging: true,
    searching: true,
    ordering: true,
    columnDefs: [
        {
            targets: 1, // colonne "Customer"
            orderDataType: 'dom-text', // trie sur le texte brut de la cellule
            type: 'string'
        },
        {
            targets: -1, // colonne "Actions"
            orderable: false,
            searchable: false
        }
    ]
});
```

> **POURQUOI PAS DE SOURCE AJAX ?**
> Avec un tableau construit par Razor (comme l'était la grille WPF via son `ItemsSource` lié au ViewModel), DataTables n'a pas besoin de requêter les données : elles sont déjà dans le DOM au premier rendu de la page. Cela simplifie le contrôleur (pas d'action `GetData` séparée) et évite un « flash » de tableau vide le temps du chargement AJAX.

5.4 Passage en mode édition

Le tableau étant rendu par Razor (pas de `columns.data` mappé), toute la logique d'édition travaille **directement sur le DOM** (jQuery classique) plutôt que sur l'objet interne de DataTables — exactement comme on manipulerait les `TextBox` d'une ligne de `DataGrid` en WPF.

```
$('#customersTable tbody').on('click', 'td.editable-cell', function () {
    const $cell = $(this);
    $cell.find('.cell-value').addClass('d-none');
    $cell.find('.cell-input').removeClass('d-none').trigger('focus');
});
```

5.5 Gestion des boutons Valider / Annuler

Les boutons apparaissent **dès la première saisie** (évènement `input`) dans un champ éditable de la ligne. La ligne modifiée est identifiée par son `data-id` (posé par Razor, cf. §5.1) et ajoutée à une liste en mémoire, en attente de la sauvegarde globale.

```
const modifiedRows = new Map(); // clé = Id de la ligne (data-id), valeur = données modifiées

// Apparition des boutons dès la saisie
$('#customersTable tbody').on('input', '.cell-input', function () {
    const $row = $(this).closest('tr');
    $row.find('.btn-validate, .btn-cancel').removeClass('d-none');
    $row.addClass('row-editing');
});

// Clic sur Valider (✓)
$('#customersTable tbody').on('click', '.btn-validate', function () {
    const $row = $(this).closest('tr');
    const id = $row.data('id');

    // lecture des valeurs saisies dans les inputs de la ligne (DOM)
    const updated = { id: id };
    $row.find('.editable-cell').each(function () {
        const field = $(this).data('field');       // ex. "Montant"
        const value = $(this).find('.cell-input').val();
        updated[field] = value;

        // affichage : on repasse en lecture seule avec la nouvelle valeur
        $(this).find('.cell-value').text(value).removeClass('d-none');
        $(this).find('.cell-input').addClass('d-none').data('original', value);
    });

    modifiedRows.set(id, updated);

    $row.removeClass('row-editing').addClass('row-pending-save');
    $row.find('.btn-validate, .btn-cancel').addClass('d-none');

    $('#btnSaveAll').prop('disabled', modifiedRows.size === 0);
});

// Clic sur Annuler (✗)
$('#customersTable tbody').on('click', '.btn-cancel', function () {
    const $row = $(this).closest('tr');
    const id = $row.data('id');

    // restaure la valeur d'origine (attribut data-original posé par Razor au chargement)
    $row.find('.editable-cell').each(function () {
        const $input = $(this).find('.cell-input');
        const original = $input.data('original');
        $input.val(original);
        $(this).find('.cell-value').text(original).removeClass('d-none');
        $input.addClass('d-none');
    });

    modifiedRows.delete(id);
    $row.removeClass('row-editing row-pending-save');
    $row.find('.btn-validate, .btn-cancel').addClass('d-none');

    $('#btnSaveAll').prop('disabled', modifiedRows.size === 0);
});
```

5.6 Sauvegarde globale

Au clic sur le bouton global, les lignes validées (map `modifiedRows`) sont envoyées en un seul appel AJAX. Comme le tableau n'est **pas** piloté par une source de données JSON, il n'y a pas de `table.ajax.reload()` possible après coup : les cellules ont déjà été mises à jour visuellement à l'étape « Valider » (§5.5) ; on se contente donc de nettoyer l'état « en attente » des lignes sauvegardées.

```
$('#btnSaveAll').on('click', function () {
    const payload = { modifiedRows: Array.from(modifiedRows.values()) };

    $.ajax({
        url: '/Customers/SaveChanges',
        method: 'POST',
        contentType: 'application/json',
        data: JSON.stringify(payload),
        success: function (response) {
            if (response.success) {
                // retire le style "en attente" des lignes désormais enregistrées
                modifiedRows.forEach((row, id) => {
                    $(`#customersTable tbody tr[data-id="${id}"]`).removeClass('row-pending-save');
                });
                modifiedRows.clear();
                $('#btnSaveAll').prop('disabled', true);
                showNotification('success', 'Modifications enregistrées.');
            } else {
                showNotification('error', response.errors);
            }
        },
        error: function () {
            showNotification('error', 'Erreur lors de la sauvegarde.');
        }
    });
});
```

ALTERNATIVE POSSIBLE

Si l'on préfère réafficher des données strictement fraîches depuis la base après sauvegarde (plutôt que de faire confiance à l'affichage local mis à jour au clic sur ✓), deux options :

> recharger la page (`location.reload()`) ou faire renvoyer par `SaveChanges` une **Partial View** Razor du tableau que l'on réinjecte dans le DOM, suivie d'une réinitialisation de DataTables (`table.destroy()` puis nouvel appel à `.DataTable()`).

6. Implémentation ASP.NET MVC (back-end)

6.1 Contrôleur

```
public class CustomersController : Controller
{
    private readonly ICustomerService _customerService;

    public CustomersController(ICustomerService customerService)
    {
        _customerService = customerService;
    }

    // Charge les données et les pousse directement dans le Model de la vue
    // (équivalent du ItemsSource de la grille WPF)
    public ActionResult Index()
    {
        var model = new CustomersPageViewModel
        {
            Site = _customerService.GetAll() // List<CustomerRowViewModel>
        };

        return View(model);
    }

    [HttpPost]
    public JsonResult SaveChanges(SaveChangesRequest request)
    {
        if (request?.ModifiedRows == null || !request.ModifiedRows.Any())
            return Json(new { success = false, errors = "Aucune ligne à enregistrer." });

        var result = _customerService.UpdateRows(request.ModifiedRows);

        return Json(new
        {
            success = result.IsSuccess,
            errors = result.Errors
        });
    }
}
```

6.2 Endpoints — résumé

Endpoint	Méthode	Entrée	Sortie
`/Customers/Index`	GET	—	Vue Razor HTML — tableau déjà rempli (`Model.Site`), **pas de JSON**
`/Customers/SaveChanges`	POST (AJAX)	JSON — `SaveChangesRequest` (liste des lignes modifiées)	JSON — `{ success, errors }`

Il n'y a donc qu'un seul point d'échange AJAX dans cette page : la sauvegarde groupée. Le chargement initial des données passe exclusivement par le rendu Razor de `Index` .

7. Règles de gestion & cas limites

- **Plusieurs lignes en édition simultanée :** l'utilisateur doit pouvoir éditer plusieurs lignes avant de cliquer sur « Sauvegarder les changements » (comportement WPF à confirmer).
- **Navigation / changement de page DataTables pendant une édition :** définir si les modifications non validées (✓) sont perdues ou conservées en mémoire par Id de ligne.
- **Concurrence :** gérer le cas où une ligne a été modifiée par un autre utilisateur entre le chargement et la sauvegarde (ex. vérification d'un timestamp / version).
- **Validation des données :** appliquer, côté client (avant apparition du bouton ✓ actif) et côté serveur, les mêmes règles de validation que l'application WPF (formats, champs obligatoires, plages de valeurs).
- **Annulation globale :** prévoir si nécessaire un moyen d'annuler toutes les modifications en attente en une seule action.

8. Checklist de recette

#	Scénario	Résultat attendu
1	Chargement de la page	Tableau affiché, aucun bouton ✓/× visible
2	Clic dans une cellule éditable	La cellule devient un champ de saisie
3	Saisie d'un caractère dans un champ éditable	Les boutons ✓ et × apparaissent sur la ligne
4	Clic sur ×	Valeur d'origine restaurée, boutons disparaissent
5	Clic sur ✓	Ligne marquée « modifiée », boutons disparaissent, bouton global activé

6	Colonne Customer	ClientId à gauche, Code client centré, Currency à droite, dans la même cellule
7	Clic sur « Sauvegarder les changements »	Toutes les lignes validées sont enregistrées en base en un seul appel
8	Erreur serveur lors de la sauvegarde	Message d'erreur affiché, lignes concernées conservées en état « modifié »
9	Tri / pagination / recherche DataTables	Fonctionnels, sans régression sur l'état d'édition en cours

---

Document de spécification à valider avec le métier avant développement — section 2.4 (comportement exact du bouton ✓) et section 7 (cas limites) à confirmer.